scai

SORBONNE CLUSTER FOR
ARTIFICIAL INTELLIGENCE

SORBONNE UNIVERSITÉ · cnrs · Inria · Inserm · utc · MUSÉUM NATIONAL D'HISTOIRE NATURELLE

INSEAD The Business School for the World · ASSISTANCE PUBLIQUE HÔPITAUX DE PARIS www.aphp.fr · cea · IRD · CIEP

Course 2 DS — Data Preprocessing

Raphael Cousin
Data Scientist
raphael.cousin@sorbonne-universite.fr

raphaelcousin.com

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

Plan

Preprocessing / Data quality

- Missing values
- Outliers
- Duplicated
- Categorial

Duration : ~15 min

Preprocessing / Data quality

- Scale
- Feature Selection

Duration : ~30 min

Preprocessing / Data quality

- Feature Engineering

Duration : ~30 min

From Raw data to production

- Notebook
- Save/load model
- Project

Duration : ~10 min

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

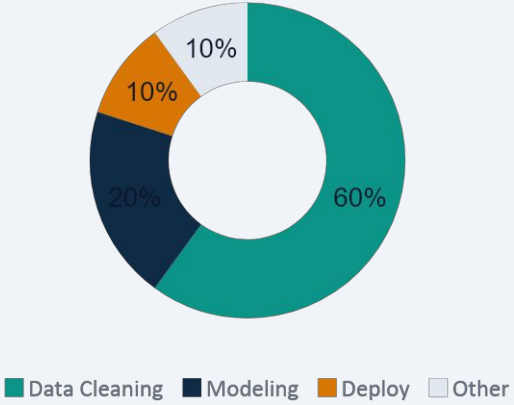
Quiz

1. 5 machine learning models you know?
2. 2 metric for regression/classification?
3. Difference between loss and metrics?
1. What is cross validation method?
2. What are the hyperparameters? Can you give me 2? How to select them?

Introduction

Preprocessing / Data quality

- Real-world data is messy: missing values, inconsistent formats, outliers, mixed types
- Models assume clean numerical input
- Preprocessing = ~80% of data science work



First Steps: Load & Inspect

```
df = pd.read_csv('data.csv')
df.shape            # rows × columns
df.dtypes           # column types
df.head()           # first rows
df.describe()       # summary
statistics
df.info()           # non-null counts
```

Common Issues

Missing values	`df.isnull().sum()`
Duplicates	`df.duplicated().sum()`
Wrong types	`df.dtypes`
Inconsistent	`df['col'].unique()`
Outliers	`describe() / boxplot`

Always inspect before anything else

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

Missing Values

Missing data

Data not collected: Information might not be collected, either due to oversight or because it was not available.

Data corruption: During data transfer or storage, data might get corrupted leading to missing entries.

	School ID	Name	Address	City	Subject	Marks	Rank	Grade
0	101.0	Alice	123 Main St	Los Angeles	Math	85.0	2	B
1	102.0	Bob	456 Oak Ave	New York	English	92.0	1	A
2	103.0	Charlie	789 Pine Ln	Houston	Science	78.0	4	C
3	NaN	David	101 Elm St	Los Angeles	Math	89.0	3	B
4	105.0	Eva	NaN	Miami	History	NaN	8	D
5	106.0	Frank	222 Maple Rd	NaN	Math	95.0	1	A
6	107.0	Grace	444 Cedar Blvd	Houston	Science	80.0	5	C
7	108.0	Henry	555 Birch Dr	New York	English	88.0	3	B

Missing data - Visualization

```
import seaborn as sns
sns.heatmap(df.isnull(), cbar=False)
```

Missing data - Strategy 1: Drop

```
df.dropna()                    # drop rows with any NaN
df.dropna(subset=['age'])      # only where 'age' is NaN
df.dropna(thresh=3)            # keep rows with ≥3 non-NaN
```

When to use

Few missing values (<5%)
Missing completely at random

Risk

Lose data
Bias if missing is not random

Missing data - Strategy 2: Imputation

Mean/Median/Mode Imputation
Missing values can often be imputed using central tendency measures such as the mean, median, or mode. This is one of the simplest methods and is particularly effective when the data does not exhibit high variance.

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
# also: 'median', 'most_frequent'
df[['age']] = imputer.fit_transform(df[['age']])
```

Mean imputation

$$\bar{X}_2 = 3.8$$
$$\bar{X}_3 = 7.8$$
$$\bar{X}_4 = 2.7$$

X_1	X_2	X_3	X_4
0	3	8	2
0	4	9	3
0	?	7	3
0	?	6	?
1	5	?	?
1	3	?	?
1	4	9	?

X_1	X_2	X_3	X_4
0	3	8	2
0	4	9	3
0	3.8	7	3
0	3.8	6	2.7
1	5	7.8	2.7
1	3	7.8	2.7
1	4	9	2.7

Missing data - Strategy 2: Forward Fill

For time series data, you might consider using methods like forward fill where you propagate the last observed data point to the next missing value.

```
# Forward fill (time series)
df['temp'] = df['temp'].ffill()
```

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

fill

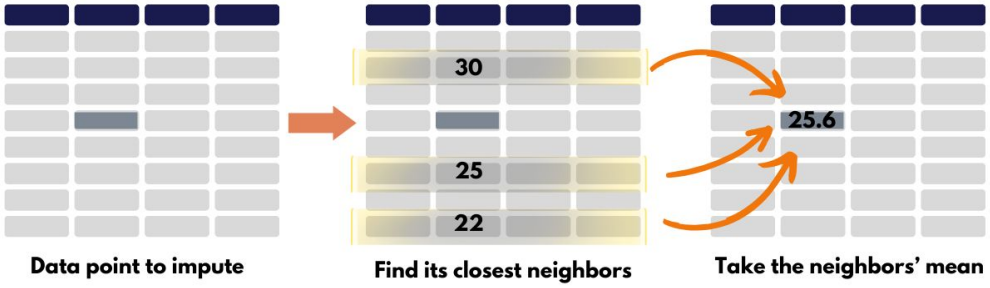
fill

fill

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	1.2
2	2021-01-03	apple	1.2
3	2021-01-04	apple	1.2
4	2021-01-01	mango	3.1
5	2021-01-02	mango	3.1
6	2021-01-03	mango	2.8
7	2021-01-04	mango	2.8

Missing data - Strategy 4: Predict

K-Nearest Neighbors
K-Nearest Neighbors can be used to impute missing values based on the similarities between features. It is more sophisticated than mean/median imputation and can provide more accurate results for complex datasets.



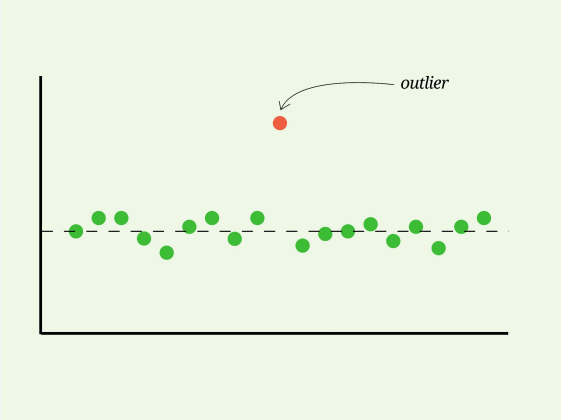

```
# Forward fill (time series)
df['temp'] = df['temp'].ffill()

# Interpolation (linear estimate between known)
df['temp'] = df['temp'].interpolate(method='linear')

# KNN Imputer (use similar rows)
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5)
df_imputed = imputer.fit_transform(df)
```

Missing data - Strategy 5: domain knowledge

Domain	Missing ≠ Random — It Means Something	Fix
Medicine	Lab not ordered → patient was fine	Impute normal, not mean-of-sick
Finance	No price → market closed	Forward-fill, never mean
Sensors	Gap → sensor died, not random	neighbor sensor
Credit	No mortgage history → never had one	Encode missingness as feature
E-commerce	No rating → didn't care enough	Implicit feedback, not ignore

Outliers

Outliers : error or signal?



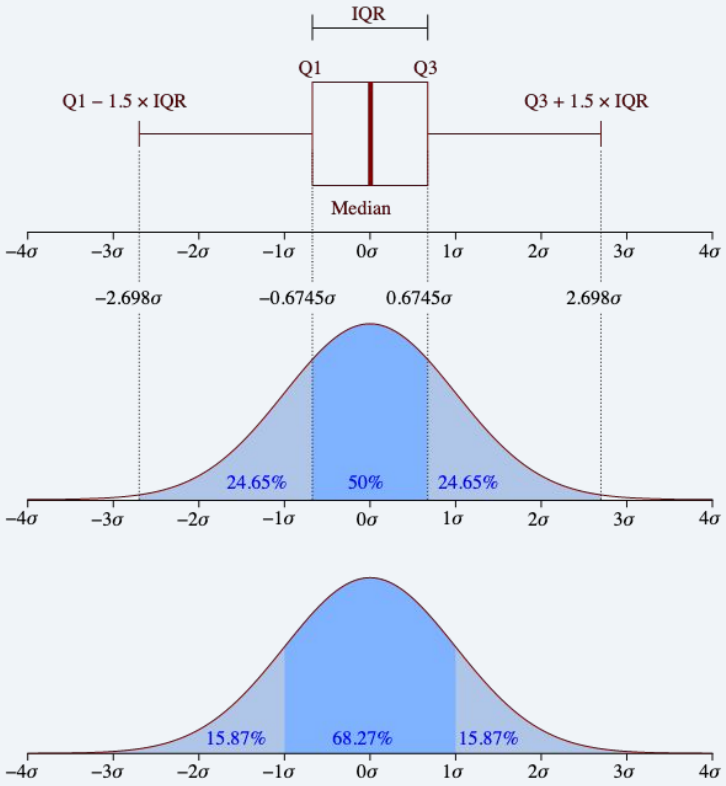
Errors

Sensor malfunction
Data entry mistake
ETL bug

Real Signal

Legitimate extreme values
Rare but real events
Important for the model

Outliers Detection: IQR Method



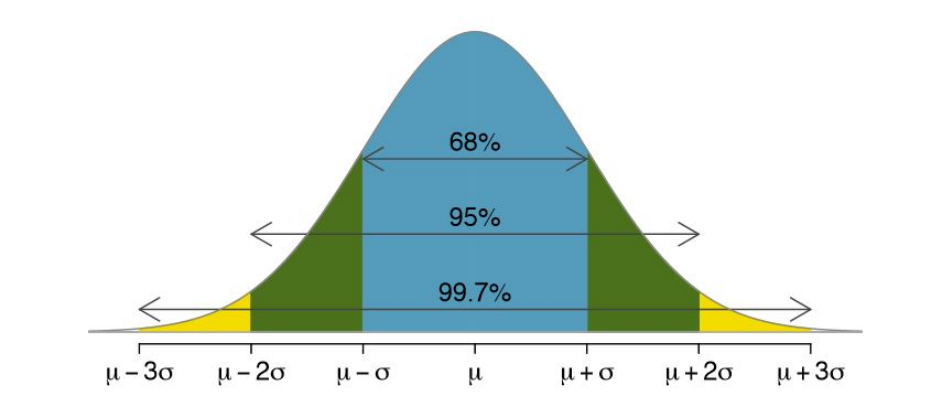

```
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df['col'] < lower)
              | (df['col'] > upper)]
```

IQR (Interquartile Range) : captures the spread of the middle 50% of your data.

Interpretation:

Large IQR ⟶ high variance/spread in the central bulk of the data. Values are dispersed widely.
Small IQR ⟶ low variance. Data points are tightly clustered around the median.

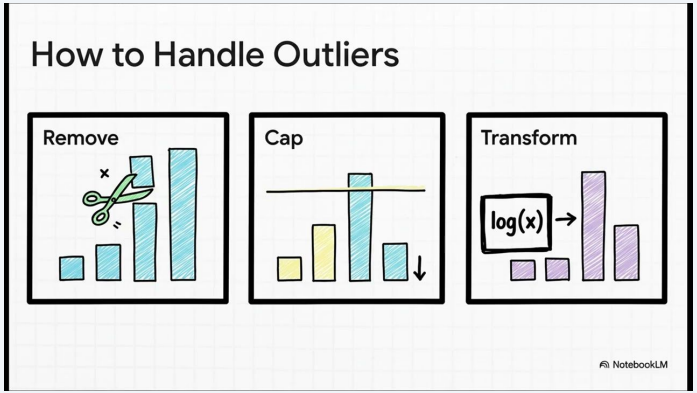
Outliers Detection: Z-Score




```
from scipy import stats
z_scores = np.abs(stats.zscore(df['col']))
outliers = df[z_scores > 3]
```

Z-score: Assumes normal distribution

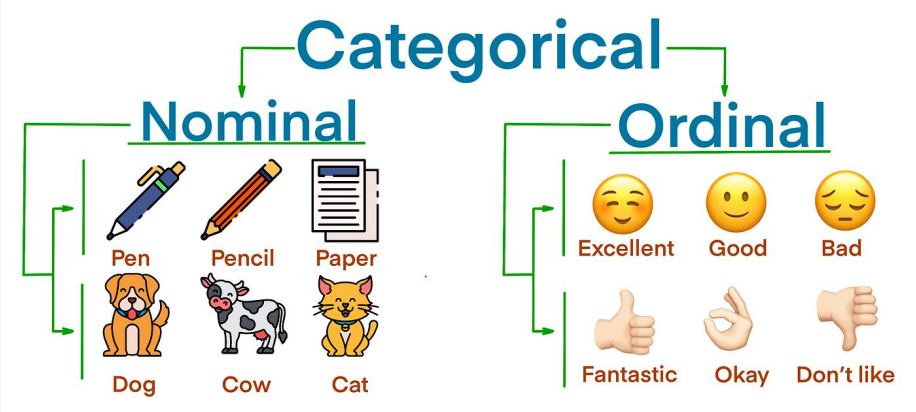
Outliers Strategy



Strategy	Code	When
Remove	df = df[~mask]	Clear errors, few outliers
Clip	df['col'].clip(lo, hi)	Limit extremes
Transform	np.log1p(df['col'])	Reduce skewness
Keep	Do nothing	Legitimate extremes

Categorical

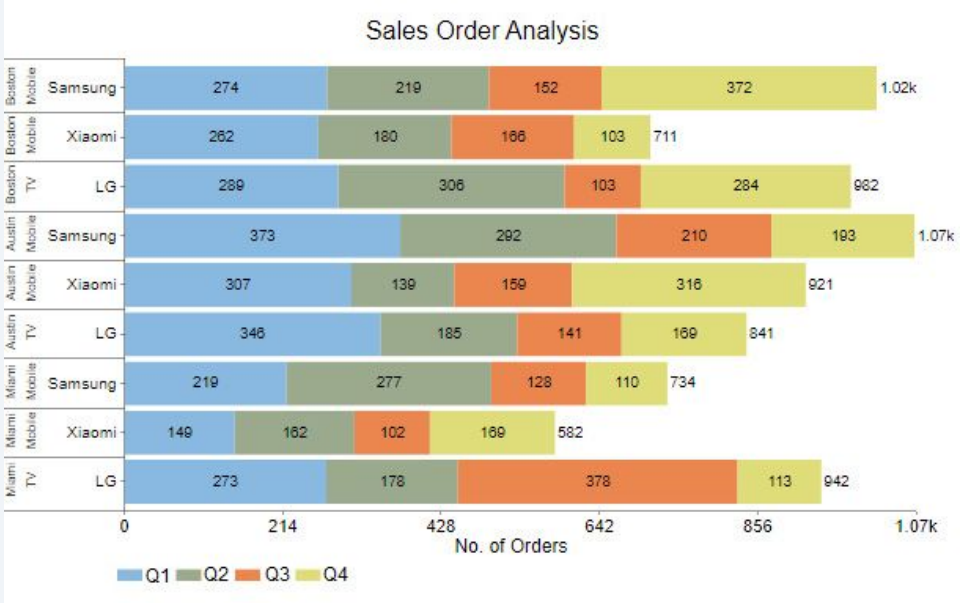
Categorical type



Nominal : Categories without inherent order
Ordinal : Categories with a logical order

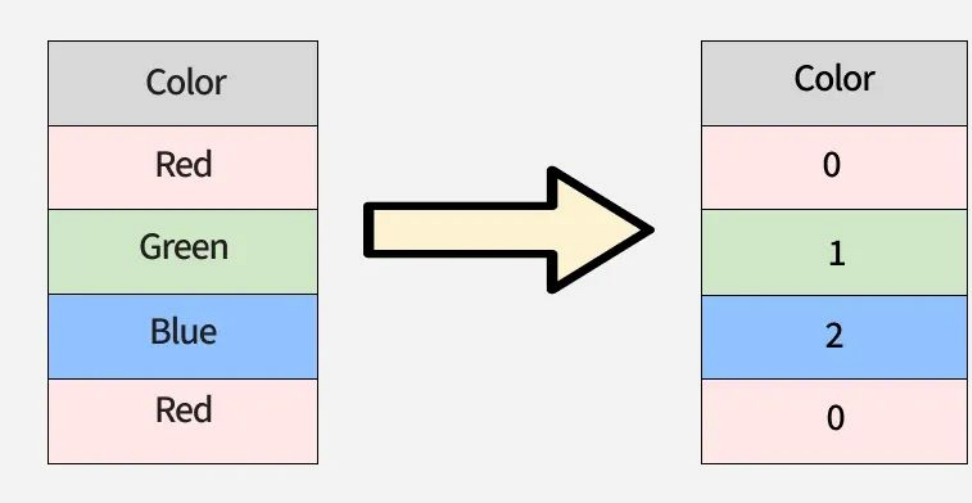
Visualize Categorical Values

```
# Count plot for 'color'
plt.figure(figsize=(10, 6))
sns.countplot(x='color', data=df, palette='viridis')
plt.title('Count Plot of Colors')
plt.xlabel('Color')
plt.ylabel('Count')
plt.show()

# Bar plot for average rating by color
plt.figure(figsize=(10, 6))
sns.barplot(x='color', y='rating', data=df, ci=None,
palette='viridis')
plt.title('Average Rating by Color')
plt.xlabel('Color')
plt.ylabel('Average Rating')
plt.show()
```



Manage Categorical Values : Label Encoding

```
le = LabelEncoder()
df['color_encoded'] = le.fit_transform(df['color'])
df['size_encoded'] = le.fit_transform(df['size'])
```



OrdinalEncoder — when categories have natural order

Manage Categorical Values : One Hot Encoding

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

→

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

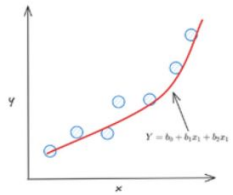
```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder()
encoded = ohe.fit_transform(df[['color', 'size']])
df_encoded = pd.DataFrame(encoded, columns=ohe.get_feature_names_out())
```

OneHotEncoder — binary columns, no ordering assumed

Feature Engineering

What Is Feature Engineering?

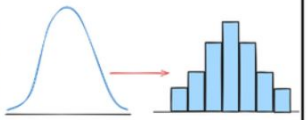
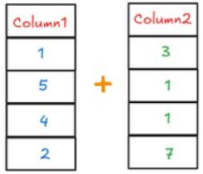
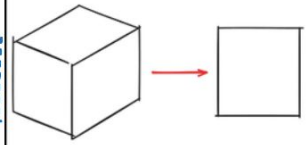
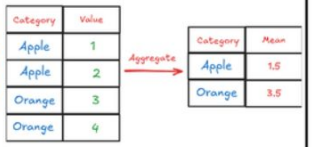
- Creating new features from existing data to help the model

		Python Implementation	Benefit
Polymonial Features		`from sklearn.preprocessing import PolynomialFeatures`	Can capture nonlinear relationships.
Target Encoding	Category / Target Apple 0 Apple 1 Orange 1 Orange 0 → Category / Target Apple 0.2 Apple 0.2 Orange 0.7 Orange 0.7	`from category_encoders import TargetEncoder`	Reduces dimensionality for high cardinality
Feature Hashing	Category Apple Apple Orange Orange → Hash 1 / Hash 2 0 0 0 0 1 0 1 0	`from sklearn.feature_extraction import FeatureHasher`	Efficient for high-cardinal features
Lag Features	Day / Value / Lag-1 / Lag-2 1 10 2 20 10 3 30 20 10 4 40 30 20	`df['lag1'] = df['feature'].shift(1)`	Capturing temporal patterns

scai SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

What Is Feature Engineering?

Often the single biggest driver of model performance .Domain knowledge is key — you know what matters

Binning		`pd.cut(df['feature'], bins=5)`	Simplifies data
Feature Interactions		`df['interaction'] = df['feature1'] * df['feature2']`	Improve model performance by capturing feature dependencies
Dimensionality Reduction		`from sklearn.decomposition import PCA`	Improve computational efficiency and reduce overfitting.
Group Aggregation		`df.groupby('category') .agg({'feature': ['mean', 'sum']})`	Improving predictive power using aggregated information

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

Time Series Feature Engineering

Data is ordered — each row depends on previous rows.
Standard ML assumes independence → need temporal features.

Type	Example	Captures
Lag	temp.shift(6)	Recent history
Rolling	temp.rolling(6).mean()	Trend / smoothing
Difference	temp.diff(1)	Rate of change
Cyclical	sin(2π·hour/24)	Periodic patterns
Cross-feature	london_temp.shift(6)	Spatial dependencies

Directly relevant to your weather prediction competition!

Lag Features & Rolling Statistics

Lag Features

```
# What was temp 1h, 3h, 6h ago?
df['temp_lag1'] = df['temp'].shift(1)
df['temp_lag3'] = df['temp'].shift(3)
df['temp_lag6'] = df['temp'].shift(6)
```

"The best predictor of the future is the recent past"

Rolling Statistics

```
# Moving avg & std (6h window)
df['roll_mean_6'] = df['temp'].rolling(6).mean()
df['roll_std_6'] = df['temp'].rolling(6).std()
```

Smooths noise, captures trend + volatility

Differences (Rate of Change)

```
df['temp_diff1'] = df['temperature'].diff(1)   # change in last hour
df['temp_diff6'] = df['temperature'].diff(6)   # change over 6 hours
```

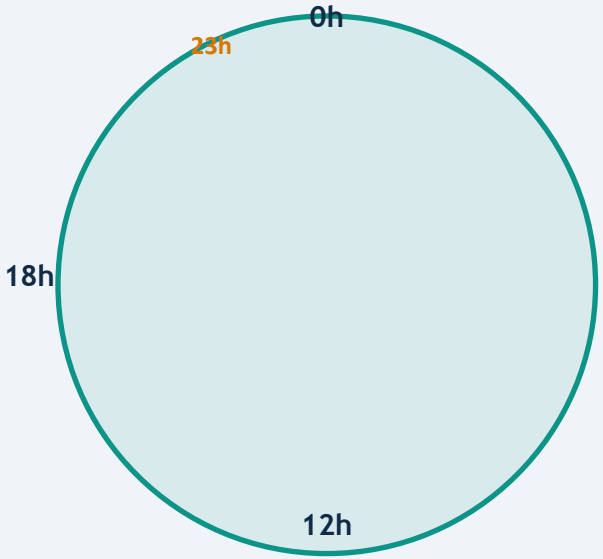
Cyclical Time Encoding

Problem: Hour 23 and hour 0 are close, but numerically far apart

```
df['sin_hour'] = np.sin(2*np.pi*df['hour']/24)
df['cos_hour'] = np.cos(2*np.pi*df['hour']/24)
```

Hour	sin	cos
0	0.00	1.00
6	1.00	0.00
12	0.00	-1.00
18	-1.00	0.00
23	-0.26	0.97

(sin, cos) space

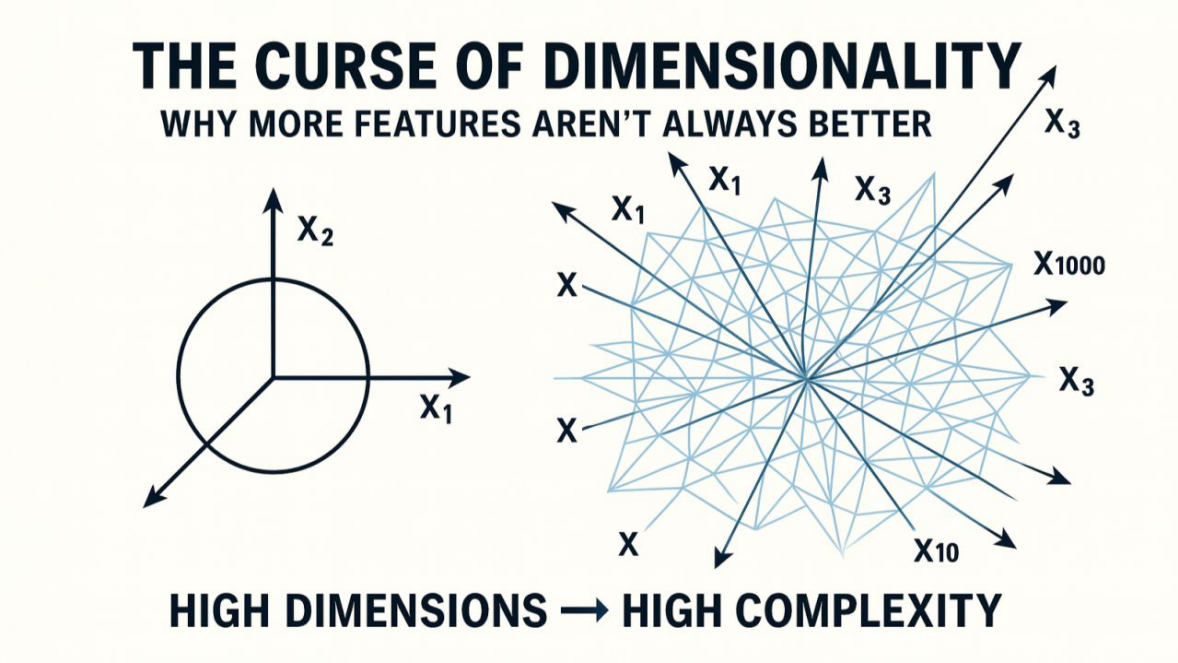


23h and 0h are now neighbors!

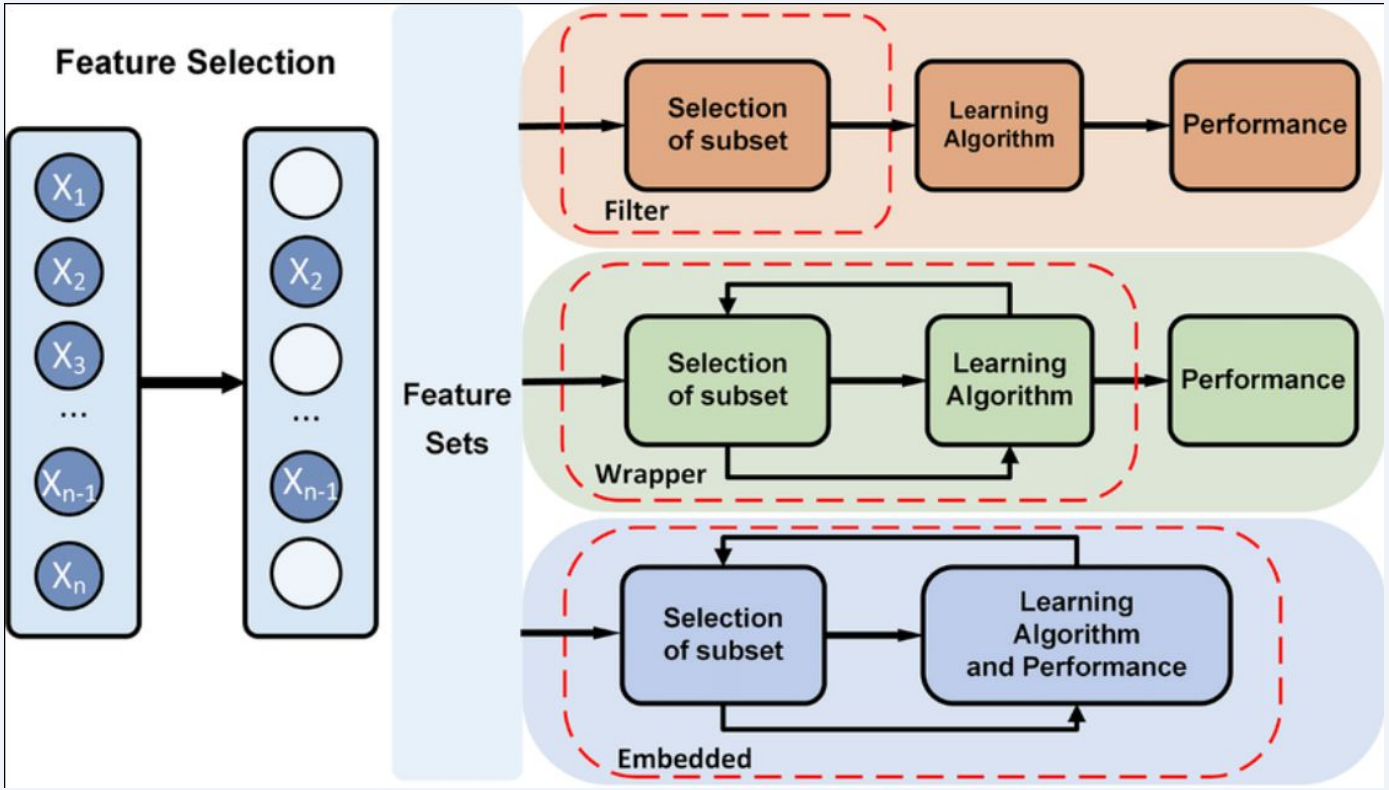
Feature Selection / Dimension Reduction

Too many features

Too many features ⟶ overfitting, slow training, noise. Simpler = better.



Feature Selection Methods



Feature Selection Codes

Filter

```
correlations = X_train.corrwith(y_train).abs()
selected = correlations[correlations > 0.05].index
```

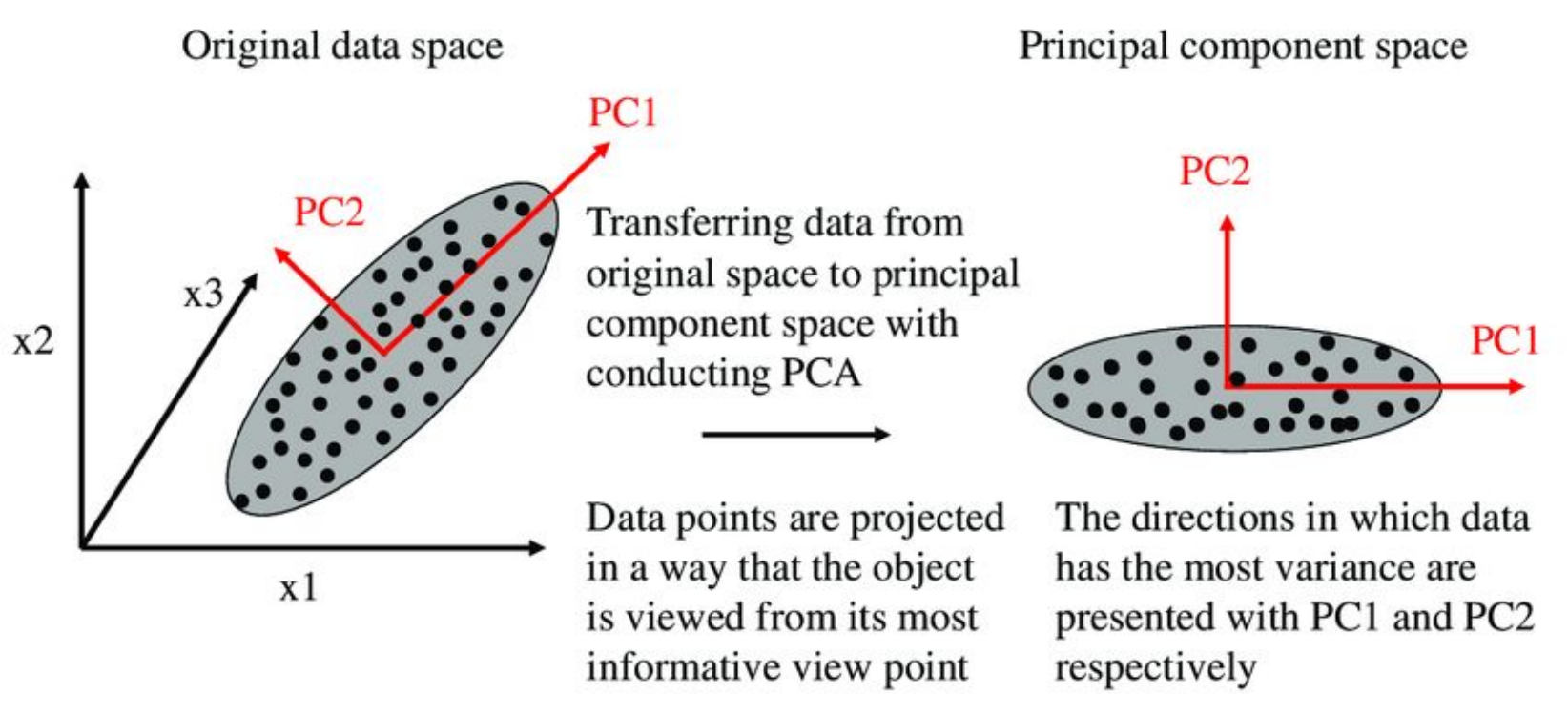
Wrapper (RFE)

```
from sklearn.feature_selection import RFE
selector = RFE(RandomForestRegressor(), n_features_to_select=10)
selector.fit(X_train, y_train)
```

Embedded (Lasso + Tree)

```
lasso = Lasso(alpha=0.1).fit(X_train, y_train)
selected = X_train.columns[lasso.coef_ != 0]  # zeros out unimportant
```

Dimension Reduction : PCA



Dimension Reduction : PCA

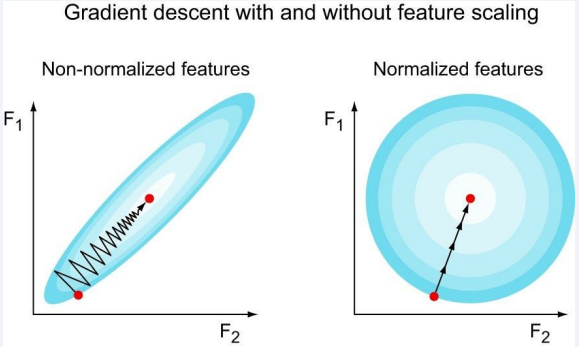
```
from sklearn.decomposition import PCA
pca = PCA(n_components=10)
X_reduced = pca.fit_transform(X_scaled)
print(f"Explained variance: "
      f"{pca.explained_variance_ratio_.sum():.1%}")
```

Scaling

Why Scale Features?

Features on different scales bias distance-based models.
Example: age (0–100) vs salary (0–100,000) → salary dominates.

Model	Needs Scaling?	Why
KNN, SVM	Yes	Distance-based
Linear / Logistic Reg.	Yes	Gradient-based
Decision Tree, RF	No	Split-based
Gradient Boosting	No	Split-based



Three Scalers

StandardScaler

`(x - mean) / std`

Best for:

General purpose

```
StandardScaler()
```

MinMaxScaler

`(x - min) / (max - min)`

Best for:

Bounded [0,1]

```
MinMaxScaler()
```

RobustScaler

`(x - median) / IQR`

Best for:

Data with outliers

```
RobustScaler()
```

Production Pipeline

Save & Load with joblib

What gets saved in the .pkl file:

Component	What's stored
StandardScaler	mean, std from training data
OneHotEncoder	category mappings
SimpleImputer	fill values (mean, median)
Model	learned weights / trees

```
import joblib

# Save the entire pipeline
joblib.dump(full_pipeline, 'pipeline.pkl')

# Later, in production:
pipeline = joblib.load('pipeline.pkl')
predictions = pipeline.predict(X_new)  # everything handled
```

Projet

Weather Prediction Competition

Competition	Predict Paris weather +6h
Enroll	ml-arena.com/ (link in TP)
Last submission	April 9, 2026
Presentations	~April 10, 2026
Final score	30-day rolling avg, May 15

Presentation Format

Teams: 2–3 students | Duration: 10 min + 5 min Q&A | Max 5 slides